

第八届 AIC 算法创新赛 · 赛题 2「AI+软件创新」

VeriFlow

VeriFlow 软件功能设计文档

规格编译、多维验证、受约束修复与沙箱判题的设计说明

赛题：第八届 AIC 算法创新赛 · 赛题 2「AI+软件创新」

文档类型：功能设计

技术架构：Next.js + FastAPI + Python 验证器 + Docker 沙箱

编制日期：2026 年 09 月

目录

- 一、总体架构设计 4
 - （一）架构原则 4
 - （二）部署形态 4
- 二、前端模块设计 5
 - （一）导航与状态 5
 - （二）出题质检与基准页 5
- 三、后端与工作流中间表示 6
 - （一）中间表示 6
 - （二）五维静态验证 6
- 四、关键业务数据流 7
 - （一）出题验证 7
 - （二）案例 4：静态通过仍被拦截 7
 - （三）选手提交 8
- 五、核心算法设计 8
 - （一）规格编译 9
 - （二）可达性与见证路径 9
 - （三）语义约束与数据流 10
 - （四）安全策略 10
 - （五）时序监视器 10
 - （六）偏序轨迹对齐 11
 - （七）影响集增量验证 12
 - （八）受约束修复 13
 - （九）反例缩小 14
 - （十）合成变异与四项计分 14
 - （十一）沙箱、对拍与参考解变异 15
- 六、门禁与证据 16
 - （一）门禁 16
 - （二）证据导出 16

- 七、评测基准在系统中的位置 16
- 八、数据对象与状态 17
- 九、安全、性能取舍与扩展边界 17
 - （一）性能取舍 17
 - （二）仍然没有的能力 17
- 十、附录 18
 - （一）附录 A：开源组件 18
 - （二）附录 B：与旧设计说明的边界 18
 - （三）附录 C：复现命令 18

一、总体架构设计

（一）架构原则

VeriFlow 把大模型出题收成一条可以否决的链：需求、规格、工作流中间表示、五维静态验证、受约束修复、模拟运行、时序监视、门禁。门禁放行之后，出题图才允许入库。选手程序走另一条链，进入 Docker 或进程沙箱，对错只由沙箱结果决定。

模型可以出现在四个位置：把题意编译成工作流、可选地提议修复补丁、在已有反例上做教练追问、以及单独的大模型对照实验。这四个位置都不读门禁的输出，也不向门禁写判定。门禁读的是静态验证结果、运行监视结果，以及策略里的严重级、未知项和覆盖率下限。模型写出的文本不进入这个判定函数。

这样做是为了把「提案」和「否决」拆开。同一个模型既生成流程图又给自己打分时，结论无法复核。VeriFlow 的判定函数对同一张图、同一份规格、同一条轨迹给出同一结果，不依赖采样温度。

（二）部署形态

浏览器访问 nginx 的 8081 端口。路径 /api 转到本机 8010 的 FastAPI，其余转到本机 3000 的 Next.js。3000 不对公网直接开放。业务数据是 SQLite。评测镜像是 veriflow-sandbox。公网演示以 Docker 为准。变量写成 docker 后，Docker 不可用就失败关闭，不会自动改成进程沙箱。进程沙箱只在变量写成 process、并且 VERIFLOW_ENV 不是 production 时启用。80 端口是主机默认站点。

前端按提交号构建到独立目录，完成后再切换 systemd 工作目录。当前发布支持开发机预构建、deploy/release.py pack 打包和服务端 install；web-release.sh 的服务器构建为备用。release-version.env 覆盖版本配置。2026-10-05 核对公网演示。当前版本以 /api/version 返回的 git_commit 和 built_at 为准；2026-09-30 的 eb94cec30307 及构建时间仅为历史发布记录。

历史版本：eb94cec30307dfcf24b45ee470c8acd617a23f7f

表 1-1 架构分层

层次	职责	位置
表现层	中文工作台、出题图、基准页	apps/web
接口层	登录、题库、提交、验证、基准	services/api
规格与中间表示	需求编译、工作流结构	packages/spec、packages/ir
验证层	五维静态检查、问题与见证	packages/verify、packages/staticcheck

运行层	模拟执行、时序监视、轨迹对齐	packages/runtime
修复层	补丁守卫、筛选、词典序、回滚	packages/repair
判题层	沙箱、对拍、参考解变异	packages/sandbox、compare、mutate
交付层	systemd、nginx、按提交号发布	deploy

二、前端模块设计

（一）导航与状态

顶栏固定六项：控制台、题库训练、沙箱判题、智能对拍、出题质检、需求出题。验证实验室包含评测基准、证据链分析、算法矩阵、系统架构、历史记录。竞赛题单包含题单分组、签到时长、括号匹配。管理员另见后台。页面只负责请求接口和管理显示状态，不在浏览器里改判题结果。

出题图使用 React Flow。做题编辑器使用 Monaco。协议状态 PASS、FAIL、READY 保留英文，其余说明用中文。控制台标题下的三项数字来自基准最新结果接口，与 experiments/runs/competition/metrics.json 同源。这三项是检测 F1、静态修复接受率、平均静态核验，分别对应「有没有抓住异常」「补丁有没有被重新检查接受」「静态检查在进程内花了多久」。

（二）出题质检与基准页

出题质检的四条案例来自 examples/golden。案例 4 的门禁状态由演示接口返回。支付分支在节点上显示原始表达式 `payment_status == success`。轨迹在最后一条已经观察到、且后继没有执行的边上标记终止。节点仍然画在图上，只说明它被画出来了；门禁看的是轨迹里有没有执行到它。「核验案例」下拉只改选项。点「运行案例」之后，图和问题列表才按这一条重跑。故事板文字跟下拉走，会比图先变。

评测基准页标题写明合成变异。消融表使用同一次运行的四行。确定性静态对照取「关闭运行监视」那一行，检测 F1 为 0.947。完整模式是 1.000。大模型对照读 llm_judge.json 的状态。没有密钥时这一行是未运行，页面不填写分数。

三、后端与工作流中间表示

（一）中间表示

工作流中间表示是出题图的规范形式，包含节点、边、类型和配置。节点种类限于工具、守卫、人工门、变换、分支、通知。领域目前有出题和校园附录。校园报销示例用来说明同一套中间表示可以换领域，附录不单独做成产品。

n8n 适配器只做节点和连线的子集往返，参数里保留种类、工具和表达式。没有基地址和密钥时标记为可选集成不可用，并走模拟运行。Dify 没有适配器。中间表示故意收窄，是为了让后面的图算法有确定的节点种类，而不是去兼容一个会随时增加节点的外部产品目录。

（二）五维静态验证

verify_workflow 五个维度是结构、语义、数据流、可达、安全。某一维没有问题时记通过，出现 CRITICAL 或 HIGH 记失败，只有 MEDIUM 或 LOW 记警告。尚未能判定写在单条约束和运行监视上，不写进这一维的汇总状态。总状态在已经失败时记失败；没有失败问题、但仍有未知约束时记尚未能判定，这时不记成通过。安全类如果出现尚未能判定，按当前策略可以拦截。运行监视的尚未能判定在默认策略里只记警告，不单独拦截。当前安全检查写出的是失败问题，不是尚未能判定。

图检查在 packages/staticcheck 中完成，不调用模型。白名单、失败处理、类型、死节点和有向环按图规模扫描，缺少审题门对每个入库节点另做一次避开搜索。设发布节点数为 P 、守卫解析和变量处理成本为 G ，总代价保守记为 $O((P+1) \cdot (V+E)+G)$ 。类型不一致归数据流，死节点归可达，缺少审题门归安全，其余结构错误归结构。

守卫表达式不交给模型去理解一句中文。parse_guard 先把 && 和 || 换成 and 和 or，再用 Python 的 ast 按表达式模式解析。单独一个名字会被拒绝，因为自然语言不是守卫。语法树允许比较、四则运算、整除、取模、乘方、正负号、与或非、下标、列表和元组，以及 min、max、len、abs。属性可以写，双下划线属性和关键字参数不行。解析通过后再收集自由变量，变量没有定义就报错。案例 4 的 payment_status == success 走的就是这条解析，页面上保留原始表达式。

死节点用一次广度优先搜索。从入度为 0 的入口出发，到不了的节点记为死节点。能到达、但没有出边、又不是入库这类终点的节点，同样记为死节点。有向环用三色深度优先搜索。节点分白、灰、黑。走到一条指向灰色节点的回边，就沿父指针还原

出一个环。自环直接给出两个相同端点。出题域禁止有向环，发现即失败。只报告一个环，不枚举全部环，复杂度仍是 $O(V+E)$ 。

四、关键业务数据流

（一）出题验证

需求文本先编译成规格。工作流进入静态验证。若案例或请求要求运行监视，则生成轨迹并对照规格。问题对象带有期望、实际、见证路径和修复提示。门禁汇总静态结果、运行结果和策略后给出放行、待复核或拦截。

（二）案例 4：静态通过仍被拦截

案例 4 的图是完整的，六个节点按生成器、范围守卫、支付状态分支、通知、审题门、入库连接。静态检查可以通过，审题门仍画在图上。分支条件是 `payment_status` 等于 `success`。演示执行在支付状态分支 `if_pay` 之后停止，这一步记成取真分支。通知、审题门和入库都没有出现在轨迹里。监视器对照这条轨迹，有五条约束失败：入库最终必须发生、入库恰好一次、跑过生成器则必须经过审题门、条件为真时必须通知、通知最终必须发生。页面把第三条显示成审题门约束未满足。入库恰好一次在列表里保留监视器写出的 `exactly once`。控制台上的案例卡片另读黄金示例里写好的一条：通知步骤未被观察到，代码是 `NOTIFY_NOT_OBSERVED`。那是卡片上的标注。出题质检页的问题列表来自当次监视，两处不是同一份列表。生成器必须先于审题门这条先后规则，在审题门一次都没有出现时记为通过，不把「没走到」记成顺序反了。门禁看到运行失败，状态是拦截。

表 4-1 案例 4 的数据流

阶段	输入	输出
静态	六个节点的完整连线	静态通过
模拟执行	在 <code>if_pay</code> 之后停止	轨迹不含通知、审题门和入库
监视	轨迹对照入库、通知和审题门	五条约束未满足
门禁	静态通过且运行失败	拦截

案例 1 与此不同。案例 1 的图只有生成器指向入库，路径上没有审题门，也没有范围守卫。静态阶段会报缺少审题门，也会报缺少上界守卫。规格还要求范围守卫和审题门至少出现一次。这两条的问题码相同、受影响节点都为空，去重后列表里留下一条，标题是规格动作未满足，描述写的是范围守卫需要至少出现一次。要补的是审

题门，不是把已有步骤的顺序对调。案例 2 是生成器输出 object、范围守卫输入 string。案例 3 的审题门仍在图上，问题是生成器配置里的 api_key 写成了明文。

案例 1 展示静态和运行监视共同拦截的结果，计数以当次图和问题为准。关闭 AI 候选后执行规则修复，最终补齐范围守卫与人工门。四节点图的 READY 只覆盖本次规格和运行条件；题包、隐藏数据及真实人工审核仍需完成。案例 4 使用完整图和模拟截断展示运行监视。

（三）选手提交

提交进入沙箱。公开样例和隐藏数据分别比对。答案错误时返回最小反例的标准输入、期望输出、实际输出和来源。对拍在生成器、暴力解和选手程序之间比较，停在第一组对不上的数据。暴力解超时单独记录，不记成选手答案错误。

五、核心算法设计

本章写的是仓库里真实运行的算法，不是界面上的说明文字。算法登记在 packages/verify/veriflow_verify/algorithms.py，页面上的算法矩阵读的是同一份登记。每一项都标明了输入、输出、复杂度和代码位置。下面先给总表，再展开难做、也最能体现本项目特点的几项。

表 5-1 核心算法

算法	做什么	复杂度
图完整性	白名单、失败处理、守卫语法、死节点	$O((P+1) \cdot (V+E)+G)$
可达性与见证	最短路径，以及避开强制检查点的路径	避开 $O(V+E)$ ；最短路含复制
规格编译	题意编译成规格，默认不调用模型	$O(\text{题意长度})$
语义约束	步骤是否存在、顺序、数据依赖	按匹配节点与搜索次数计
数据流	端点类型一致，生产者能到达消费者	$O(E)$ 加匹配节点对搜索
安全策略	明文密钥、过松范围、外发地址	$O(\text{节点数} \cdot \text{配置长度})$
时序监视	对轨迹做一遍确定性检查	$O(\text{事件数} \cdot \text{约束数})$
轨迹对齐	偏序上的一次线性扩展，再做编辑距离	$O(V(V+E)+V^2 \log V+n \cdot m)$
影响集增量验证	只重跑受改动影响的维度，大改退回全量	含排序与类别重算
补丁守卫与选择	非法补丁不落地，最多三份按词典序选取	$O(K \cdot \text{全量验证})$ ， $K \leq 3$
反例缩小	沿见证路径裁掉无关节点	$O(V+E)$
合成变异	在正确底图上按清单注入带标签的故障	按清单枚举

（一）规格编译

默认规格编译器是启发式的。它扫描题意里的关键词，生成 workflow 规格：必须出现的步骤、先后约束、数据依赖、安全策略和时序约束。这条路径的复杂度随题意长度线性增长，不访问网络，没有密钥也能运行。它的输出是给监视器和门禁用的约束，不是一段自由文本。

另一条路径把自然语言编译成 workflow 图，可以调用 DeepSeek 的 JSON 接口。没有密钥，或返回无法解析时，退回关键词示例。这条路径只负责提案。编译出来的图仍然要经过同一套验证器。模型没有返回「通过」的权力。

（二）可达性与见证路径

缺少审题门需要给出绕过人工门的见证。`bypass_path` 用父指针保存搜索树，单次复杂度 $O(V+E)$ 。`shortest_path` 按广度优先次序寻找最短路径，但当前实现入队时复制整条路径，最坏另有 $O(V^2)$ 的路径复制成本。后者不能无条件标为线性。

这里刻意不用枚举全部简单路径。稠密有向图上的全路径数量会指数增长。最短路和「避开检查点的一条路径」已经足够充当见证，也足够让修复器知道该往哪里补节点。算法登记里写明了这个边界：不在稠密图上枚举全部路径。

避开检查点的实现是带障碍的广度优先搜索，仍然是 $O(V+E)$ 。检查点先标成不可进入。目标本身若是检查点，直接没有路径。源点只取入度为 0、并且自己不是检查点的节点。队列按层扩展，已经看见的节点和检查点都不再入队，父指针用来回溯。缺少审题门时，检查点是图上全部人工门，目标是每一个 `publish_problem`。案例 1 里没有人工门，生成器入度为 0，它的后继入库不是检查点，于是命中，回溯得到生成器到入库。顺序约束用的是同一个函数：把必须先发生的节点当成检查点，若仍能走到后继，这条路就是顺序被绕开的见证。若绕不开，再看有没有一条从前者到后者的最短路。有绕开的路，或者从前任走不到后继，都算这条顺序约束被违反。只有绕不开、并且确实存在从前者到后者的路，才算没有被违反。走不到时，见证只记下后继节点自己。

图模块里还有一个上限为 16 条简单路径的深度优先搜索。解释模块里的函数可以用它挑一条不经过必经节点的路径。这条函数目前没有被门禁、静态检查或页面调用。缺审题门的见证用的是上面那条线性时间的避开搜索，避免路径条数随图变密而指数增长。

（三）语义约束与数据流

语义检查按节点标识、种类或工具名匹配，再检查基数、先后约束和数据依赖。选择器可匹配多个节点，搜索次数依匹配组合而定。设约束数为 C 、实际搜索次数为 Q ，保守代价为 $O(C \cdot V + Q \cdot (V + E + V^2))$ ； V^2 项来自当前最短路的路径复制。

数据流先以 $O(E)$ 比较边两端声明的类型，再按规格检查生产者是否能到达消费者。案例 2 为生成器输出 `object`、范围守卫输入 `string`。每条依赖可能匹配多个生产者和消费者，后续搜索按实际节点对计，并包含最短路的路径复制成本。当前不做字段级切片或完整污点传播。

（四）安全策略

安全策略函数扫描节点配置，识别硬编码密钥、过松的数据范围，以及指向外网的 URL。缺少人工审题门不在这次配置扫描里产生。它由图上的避开搜索发现，再归入安全类。案例 3 的审题门还在图上，失败点是生成器配置里的 `api_key`。配置扫描的代价与节点数和配置文本长度成正比。

这项检查提供的是风险记录。它不构成形式化证明，也不是到达外部出口的完整污点分析。证据导出里同样声明该文件不是形式化证明。文档和页面对这一边界使用同一句话，避免把启发式扫描说成定理证明。

（五）时序监视器

静态图回答「画了没有」，监视器回答「轨迹里走到没有」。模拟执行可以在指定节点之后停止，也可以指定分支走向。监视器把规格编译成一组规则，对轨迹做一遍扫描，复杂度 $O(\text{事件数} \cdot \text{约束数})$ 。它不是把线性时序逻辑展开成自动机。支持的规则是项目里真正会用到的一个子集，因此可以在一次扫描里判定，而不必构造乘积自动机。

已实现的规则包括：某步必须先于另一步；某步最终必须出现；某步恰好出现一次、至少一次或至多一次；某步执行了则另一段必须随后出现；某分支取到指定值则后续必须出现；消费者执行前生产者必须已经出现。完成事件都计入次数。轨迹里没有单独的开始事件，若按节点去重，重试和重复写入会被藏掉，所以实现按事件序号计数。

分支规则有一个刻意的未知状态。若轨迹里没有观察到指定分支，结果是尚未能判定，而不是失败。观察不到和观察到了但后续没出现，是两件不同的事。每条分支

触发都会留下一个义务：后续必须出现约定节点。一次更晚的后续事件可以同时了结多个尚未了结的义务，但不能了结发生在它之后的触发。

案例 4 用的是「在支付状态分支之后停止」。图上的通知、审题门和入库都还在，轨迹里没有它们。支付分支这一步已经记成取真，所以「条件为真则必须通知」记失败，不是尚未能判定。入库最终出现、入库恰好一次、跑过生成器则必须经过审题门、通知最终出现，这几条也会失败，并带上违反位置和轨迹切片。门禁看到运行失败，即使静态是通过，也拦截。

（六）偏序轨迹对齐

出题图常常有汇合和分支，期望执行不是唯一的一条全序。若把图强行排成一个序列再和轨迹比编辑距离，并发节点对调顺序会被误报成乱序。对齐算法先承认偏序，再做序列比对。

第一步，对每个节点分别搜索后继，得到 **happens-before** 可达关系，最坏 $O(V \cdot (V+E))$ ；规格先后约束再补充匹配节点对。第二步，**Kahn** 过程按观测位置反复排序就绪节点，当前最坏有 $O(V^2 \log V)$ 排序成本。第三步，用编辑距离动态规划比对期望与观测序列，时间和矩阵空间 $O(n \cdot m)$ 。这里分别陈述实际实现的各项成本。

编辑距离的递推和经典的 **Needleman-Wunsch** 相同，但替换代价设为很大的常数，只有标识相同才允许零代价对角前进。动态规划因此只在两条序列之间做缺位匹配，不会把两个不同标识换成彼此。缺一个期望节点，标成缺失。多一次观测执行，标成意外执行。第四步回看匹配对：若后一个节点按 **happens-before** 必须先于前一个节点，却在轨迹里更早出现，就把这一对改标为乱序。没有先后关系的两个节点对调顺序，仍然保持匹配。

期望序列长度记为 n ，观测序列长度记为 m 。替换代价写成 **subst**。标识相同是 0，否则是 1000000。对角步因此几乎不会被选中，除非两个位置是同一个节点。

```
dp[i][0] = i
dp[0][j] = j
subst = 0 if expected[i-1] == observed[j-1] else 1000000
dp[i][j] = min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + subst)
```

第一项删掉一个期望节点，标成缺失。第二项表示观测里多了一次执行，标成意外执行。第三项只在标识相同时代价为 0。回溯从右下角走到左上角，标识相同并且代价没有增加时优先走对角。以期望序列 **Schedule**、**HTTP**、**IF**、**Email** 和观测序列 **Schedule**、**HTTP**、**Email** 为例：前两个标识零代价对齐，**Email** 与 **Email** 对齐，**IF** 只

能走删除，代价为 1，标成缺失。把 IF 替换成 Email 的那一步代价是一百万，动态规划不会选它。这个常数就是对齐算法的优化：用一次 $O(n \cdot m)$ 的编辑距离完成匹配，同时不把不同节点当成可以互相替换的近似。

展示用的是一条线性扩展，不是所有交错的枚举，也不是 Petri 网过程挖掘。算法登记和返回结果里都写了这个限制。代价是并发匹配是近似的；收益是比对可以在 $O(n \cdot m)$ 内完成，并且不会把合法的并发对调报成缺陷。

（七）影响集增量验证

修复环里一张图会被反复检查。若每次都无条件跑完全部维度，候选一多就会把时间花在与本次改动无关的规则上。增量验证先对前后两张图做结构差分，再决定重跑哪些维度。

差分识别九类记录：节点增加、节点删除、节点种类或工具变化、条件表达式变化、配置变化、输入输出类型变化、触发器变化、边增加、边删除。触发器变化不是单独去扫一个触发器字段。某个节点在前后两张图里都是入口，并且它的种类或工具变了，就在种类变化之外再记一笔。节点增删、种类变化和这一笔触发器变化，都会让五个维度全部重跑，并退回全量。边的增删同样凑满五个维度，因此也退回全量。边的变化要包含安全维，是因为「缺少审题门」虽然归在安全类，它的证据来自图的连通。只改配置时重跑安全维。只改条件时重跑结构、语义和安全。只改类型绑定时重跑数据流和语义。改动对不上这些类型时，退回全部维度。

影响集沿出边搜索下游节点，记录相关约束。实际执行按受影响类别筛选：`verify_scoped` 仍在整张图和完整规格上重跑选中的类别，影响节点和约束清单用于记录及统计，不直接限制计算范围。未重跑类别保留旧问题。拓扑改动或没有旧结果时退回全量；差分和结果清单还有排序，最坏整理成本 $O((V+E) \log(V+E))$ 。

测试将增量和另一次全量结果比较，当前比较字段为总状态、问题码和失败约束，没有逐项比较所有见证、提示或轨迹。竞赛基准加速仍未测量。`incremental/latency.json` 仅是一条四节点记录，不能推广为普遍加速结论。

表 5-2 改动类型与重跑范围

改动	重跑哪些维度	为什么这样收
节点增删、种类或工具变化、触发器变化	五个维度，并退回全量	拓扑变了，旧结论不能局部拼接
边增加或删除	五个维度，并退回全量	缺少审题门来自连通，边一变就凑满五维

只改配置	安全	明文密钥和范围写在配置里
只改条件表达式	结构、语义、安全	守卫语法和策略都读表达式
只改输入输出类型	数据流、语义	类型和数据依赖跟着绑定走
没有上一次全量结果	全量	没有旧结论可保留

（八）受约束修复

修复不是让模型重画一张图再看顺眼不顺眼。受约束修复调用 `verify_repair_loop`。每一轮先为当前问题生成候选补丁。规则候选由问题码直接构造。可选的模型提议最多两条，而且只能使用允许的操作：增删节点、改参数、替换操作、连线或断线、改条件、改绑定。没有密钥时模型提议不可用，这一轮只留下规则补丁。模型提议即使存在，也不自带通过标记。

缺审题门的规则补丁找到入库节点，插入人工门或接回已有人工门，把入库的入边经过该门。案例 1 还缺范围守卫，修复环继续处理对应问题；最后形成四节点图，以重新验证为准。中间候选接受只表示目标改善且未触发回退，竞赛基准成功另以最终静态 PASS 计数。

每轮最多保留 3 份候选，避免搜索随问题数膨胀。每份候选按固定流水过关。先检查补丁结构是否合法，再检查图完整性，再检查工具是否在领域白名单里。非法补丁不会被应用到图上，记为守卫拒绝。通过守卫后先做上一节的增量筛选：目标问题被消除，或问题数下降，或失败约束数下降，才算有希望。没有希望的候选不到全量验证。有希望的候选再跑一次完整的 `verify_workflow`。接受条件是：目标问题消失，不新增高严重级问题，可达维不从无问题变成有问题，结构维也不新出现问题。否则记为没有改进或发生回退。

表 5-3 一份候选要过的关

顺序	检查	不过时怎么处理
1	补丁结构	空补丁直接拒绝
2	能否应用到当前图	不修改原图
3	工具是否在领域白名单	领域外工具拒绝
4	增量筛选	目标、问题数、失败约束都没变好，就不做全量
5	全量验证	用完整的五维结果，不用筛选结果当最终结论
6	是否变好	目标仍在、新出现高严重级、可达或结构从无问题变成有问题，都拒绝

多份候选都通过时，按词典序取最小的一份。比较键依次是：目标是否修好、新出现的高严重级问题数、失败约束数、可达维是否退化、改动的节点数、改动的边数、改动的参数数、补丁条数。排在前面的项权重大。因此在同样修好目标的前提下，改动更小的补丁胜出。这是对修复的优化：不追求模型叙述得多完整，而追求验证变好且扰动最小。

循环默认最多三轮。竞赛基准里最多两轮。若本轮问题指纹和之前某一轮相同，说明没有新进展，直接停止。任何拒绝都保留原图。另一条路径按题意重新编译整张图，用在出题草稿的「按原文再生成」。静态修复接受率只统计受约束补丁。整图重编译不进入 0.844 这个比例。

运行时专有故障不进入修复分母。静态修复环看不到「在某节点后截断轨迹」或「强行走某分支」，因为这些信息在执行参数里，不在图的补丁语言里。50 条故障中有 5 条属于这一类，分母是 45。其中 38 条被接受，7 条被拒绝，接受率 0.844。拒绝明细是：六条补丁没有使结果变好，一条补丁本身不合法。页面列出原因，不把拒绝改写成成功。

（九）反例缩小

验证失败后，若把整张图和全部节点都丢给用户，定位成本会留在人身上。反例缩小不做多轮删除再复验。它沿见证路径和受影响节点取切片。见证不少于两个节点时，展示里优先留下路径两端和受影响节点，其余节点不放进这一份结果。图上的边只保留两端都还在切片里的边。问题对象上另外标出的受影响边，即使某一端已经被裁掉，也会附在这份结果后面。这一步是线性扫描，不重新跑验证器，所以不能声称删掉任何一个节点之后问题都还在，也不声称全局最小。返回值里的 `globally_minimal` 固定为假。选手侧的最小反例是另一件事：沙箱在测资上找到的最短输入、期望输出和实际输出。

（十）合成变异与四项计分

竞赛基准不用随机抽样。五条底图分别是线性出题、双路汇合、带类型的数据流、安全策略、分支通知，对应五种拓扑。每条底图先作为未注入故障的样本。底图若静态不通过，或运行监视失败，基准脚本拒绝写出结果。然后按故障清单做确定性枚举，改坏图中的门、类型、绑定、分支或密钥。另有 5 条运行时故障，用截断或分支选择构造。合计 55 组。种子 20260919 只标记这批结果来自哪一次运行，不参与抽样。

计分把四件事分开，后一件比前一件更具体。检测：故障样本上出现任何异常，记为抓住；正常样本上出现异常，记为误报。检测 F1 由精确率和召回率合成。诊断：静态故障要求问题码等于注入时期望的码；运行时故障要求失败文本里出现约定短语。定位：静态故障要求问题的节点、边或见证路径碰到被改动的位置；运行时故障要求失败项点到被截断的节点。点不到时，这次只算检测到，不算定位成功。修复接受率只在可修复的 45 条上计算。

这 55 组上，完整模式的检测精确率、召回率和检测 F1 都是 1.000，诊断 1.000，定位 0.700，也就是 50 条里 35 条点到了改动位置。检测 F1 为 1 只说明这 55 组：50 条故障都被报成异常，5 条正常流程都没有被误报。它不是题库的通过率，也不是对未来出题图的保证。

消融用同一次运行、同一批案例，每次只关掉一部分能力，用来看每一层的贡献。只做结构、关掉语义数据流安全和运行监视时，检测 F1 为 0.462，召回率 0.300。去掉安全维但保留运行监视时，检测 F1 为 0.889，召回率 0.800。五个静态维都开、运行监视关闭时，检测 F1 为 0.947，召回率 0.900，漏掉的是 5 条只有运行才看得见的故障。四行的精确率都是 1，分数差来自漏报。基准页上的确定性静态就是 0.947 这一行。

大模型对照的输入只有需求和完整工作流。载荷不包含故障名、类别、期望码、被改节点、截断点和提示。运行时专有故障不进入这组对照，因为只看图观察不到截断。没有密钥时状态是未运行，重复次数为 0，不产生分数。llm_judge.json 里另有一份静态切片：去掉 5 条运行时故障后，剩下 45 条故障在静态检查下检测 F1 为 1。那一份只给模型对照作参照，模型本身没有运行。它和「全部 55 组关掉运行监视」的 0.947 不是同一个分母。

（十一）沙箱、对拍与参考解变异

选手程序的判定在沙箱里完成。公网使用禁网的 Docker 镜像。编译错误、答案错误、超时、内存超限、运行错误和通过分开记录。答案错误时保留一组最小反例的三列。对拍由环境执行数据生成器和一份暴力解，停在第一组不一致的地方。暴力解超时记为对拍错误。

参考解变异打在 Python 参考程序的语法树上，例如比较符和范围边界，用来计算变异杀死率。C++ 语法树变异没有做。特殊评判、交互题和 Java 没有做。这些边界写在算法限制和测试报告里，不写成已完成能力。

六、门禁与证据

（一）门禁

veriflow gate 先收集拦截原因。问题严重级达到策略中的 CRITICAL 或 HIGH、运行结果为失败、安全类问题被标成尚未能判定且策略把安全未知记为失败、或者运行约束覆盖率低于 0.9，状态都是 BLOCKED，退出码为 1。没有这些原因时，若静态总状态或运行状态是尚未能判定，状态是 REVIEW REQUIRED，退出码仍是 0。否则是 READY，退出码也是 0。退出码 2 不是门禁判定。命令行捕获到异常时用它表示工具错误。策略文件是 examples/veriflow-policy.yaml。默认把运行监视的尚未能判定记为警告，不单独拦截；安全未知记为失败。当前安全检查写出的是失败问题，不是尚未能判定。模型输出不作为门禁输入。出题人点批准时，若静态检查仍有错误，或门禁不是 READY，或整次验证失败，接口拒绝放行。选手提交状态由沙箱写回，前端不在本地改判定。

（二）证据导出

证据链分析可以导出 JSON 和 Markdown。包内有 workflow 哈希、运行编号、时间、来源需求、规格、静态问题与见证、运行状态与轨迹、门禁、修复记录和模型调用痕迹。文件声明不是形式化证明。密钥不进入导出文件。哈希和运行号用来把一次页面上的结论对应回当时的输入，而不是只留一张截图。

七、评测基准在系统中的位置

基准脚本是 python scripts/competition_benchmark.py。结果目录是 experiments/runs/competition/，生成时间 2026-09-19T07:02:56Z，数据集名称 competition-v2。平均静态耗时 0.175 毫秒，平均运行监视耗时 0.114 毫秒，都是进程内计时，不含打开网页的等待。控制台把平均静态核验显示成一位小数，那一格是 0.2 ms。experiments/runs/smoke/ 里的小样本只用于冒烟，主结果以竞赛目录为准。

基准页、控制台和本章使用同一份结果文件。文档不手填满分。若页面故事和结果文件冲突，以结果文件和案例源文件为准，并修改页面。

八、数据对象与状态

表 8-1 核心对象

对象	关键内容	设计说明
WorkflowIR	节点、边、类型、配置	出题图的规范形式
WorkflowSpec	由需求编译出的约束	监视器和门禁的依据
Issue	码、严重级、见证、受影响节点	静态或运行问题
RepairReport	每轮接受或拒绝的原因	接受率的原始记录
Evidence bundle	哈希、运行号、门禁、轨迹	导出时声明不是形式化证明
Submission	语言、判定、反例	沙箱结果

出题项目的流程状态是已检查、已阻断、审题通过、已入库。审题状态另记为待审题、已通过或已驳回。页面上的草稿是这份需求的工作副本，不是上面四个流程状态之外的第五个取值。登录后的接口校验角色。管理接口要求管理员。不能停用自己，也不能停用最后一个仍可用的管理员。演示口令来自环境变量，缺省为 `demo`、`setter`、`admin`，只用于公网演示。

九、安全、性能取舍与扩展边界

（一）性能取舍

本项目的优化都放在保证可复核的前提下。时序规则收成一次扫描，而不是通用线性时序逻辑。轨迹对齐只用一条偏向观测的线性扩展。增量验证在拓扑大改时退回全量，并用测试锁住与全量结果的等价。修复先用增量筛选丢掉没有希望的补丁，再用词典序偏好更小的改动，每轮候选不超过 3 份。可达性用最短路和一条避开检查点的路径，不枚举全部路径。这些选择的共同标准是：省掉的工作不能改变通过或失败的结论。加速倍数没有写成竞赛成绩。

（二）仍然没有的能力

SMT 或 Z3 求解、Dify、真实 n8n 控制面、特殊评判、交互题、Java、C++ 语法树变异都还没有做。安全检查和证据包都不是形式化证明。大模型对照在没有评测密钥时保持未运行。扩展时可以更换 OpenAI 兼容的模型地址，判定链不需要跟着更换。

十、附录

（一）附录 A：开源组件

Python、FastAPI、Pydantic、Uvicorn、Next.js、React、Monaco、React Flow、Docker、nginx、pytest 按各自上游许可证使用。仓库中有一份 React Bits 自由组件的本地改编，许可证是 MIT 加 Commons Clause。改编清单见仓库根目录 THIRD_PARTY_NOTICES.md。题面为原创。

（二）附录 B：与旧设计说明的边界

docs/superpowers/specs/2026-09-10-veriflow-design.md 是 9 月 10 日的实现计划。当前行为以本文件、packages 源码和 experiments/runs/competition 为准。修复的主路径是受约束补丁。公网入口带 8081 端口。后端测试见测试报告中的 287 项。

（三）附录 C：复现命令

```
python -m pytest -q --tb=no
python scripts/competition_benchmark.py
python -m veriflow_cli verify --workflow examples/compose/missing_gate.json --nl "完整出题。"
--explain
```